Project Report: SecureShield - Flask Authentication System

Team Members:

- Sumayo Omar (ID: 230208934)
- Hafsa Liban (ID: 230208941)

1. Introduction

Modern web applications demand robust authentication and authorization to protect user data and control access to resources. The **SecureShield** project addresses this by developing a secure backend web application using Python Flask. It implements a comprehensive authentication flow and Role-Based Access Control (RBAC) to regulate access based on user roles, simulating a real-world secure backend environment.

2. Objectives

The primary objectives of SecureShield are:

- Implement secure user registration with hashed passwords.
- Develop a login system issuing JSON Web Tokens (JWT) for stateless authentication.
- Enforce Role-Based Access Control (RBAC) for feature restriction based on Admin/User roles.
- Secure API endpoints, ensuring only authenticated and authorized access.
- Implement token revocation (logout) via a blacklist for secure session invalidation.

3. Tools and Technologies

- **Python:** Core programming language.
- **Flask Framework:** Lightweight web framework for RESTful API development.
- **Flask-Bcrypt:** Provides bcrypt hashing for secure password storage.
- **PyJWT:** Python library for encoding/decoding JSON Web Tokens.
- **Postman:** Used for API testing and verification of authentication flows.

4. System Design

SecureShield employs a client-server architecture with a Flask backend acting as a RESTful API. The system integrates Authentication & Identity with Access Control & Authorization.

- **Authentication:** User registration involves bcrypt hashing of passwords before storage. Upon login, credentials are verified, and a JWT containing user identity and role is issued.
- **Authorization:** Protected routes require a valid JWT in the `Authorization` header. A custom middleware validates the token (signature, expiration, blacklist status) and verifies the user's role before granting resource access.

5. API Endpoints

Endpoint	Method	Access Level	Description
/register`	POST	Public	Creates new user accounts with hashed passwords.
/ login`	POST	Public	Authenticates users, returning a signed JWT on

			success.
/profile`	GET	User & Admin	Protected route for user profile information, requires valid JWT.
/ user/<username>`	DELETE	Admin Only	Highly protected route for deleting user accounts, requires Admin role.
/logout	POST	Authenticated	Invalidates the current JWT by adding it to a blacklist.

6. Security Features

SecureShield incorporates critical security measures:

Password Hashing and Salting

Passwords are never stored in plain text. `bcrypt` is used to salt and hash passwords. **Salting prevents "Rainbow Table" attacks** by adding a unique random string to each password before hashing, making precomputed hash tables ineffective against brute-force attempts.

JSON Web Tokens (JWT)

JWTs enable stateless authentication. The token payload contains non-sensitive identifiers (username, role). It is crucial to understand the **risks of storing sensitive data (like passwords) inside a JWT payload**. JWT payloads are Base64Url encoded, not encrypted, meaning their contents are easily readable if intercepted. SecureShield avoids this risk by only including non-confidential claims.

Role-Based Access Control (RBAC)

Strictly adheres to the "Principle of Least Privilege." Standard users are denied access to administrative routes (e.g., DELETE), preventing unauthorized privilege escalation and ensuring actions align with assigned roles.

Token Blacklisting

To securely implement logout for stateless JWTs, a token blacklist is used. Upon logout, the token is added to this list, and subsequent requests using that blacklisted token are rejected by the middleware, even if the token has not naturally expired.

7. Testing with Postman

Postman was used for rigorous API testing:

1 Registration & Login: Verified successful account creation and JWT issuance.
2 Protected Routes: Confirmed access to `/profile` with a valid JWT.
3 Role Verification: Tested `DELETE /user/<username>` with a standard user, confirming `403 Forbidden` response for unauthorized access.
4 Logout:Ve rified token blacklisting by attempting to use a logged-out token, resulting in denied access.
5 Tamper Test: Attempted to alter the JWT role (e.g., User to Admin) using `jwt.io`. Verified server rejection due to invalid signature, confirming token integrity protection.

8. Conclusion

The SecureShield project successfully implements a secure backend authentication system using Python Flask. It effectively protects sensitive endpoints through JWTs for stateless authentication, bcrypt for secure password storage, and robust Role-Based Access Control. The project underscores the importance of the principle of least privilege and secure data handling, providing a reliable and secure foundation for modern web applications.